

SBA301

BUỔI 04 / SESSION 04

Props, State, Hooks & Context

Dynamic React Components

MỤC TIÊU CHUYỂN ĐỔI TƯ DUY Từ “UI có cấu trúc component” sang “UI có dữ liệu truyền vào, có trạng thái thay đổi và có tương tác”.

SESSION 03 SESSION 04
I can structure UI. → I can make component data flow and change.

Parent → Props → Child
User Action → Event → State Update → Render → Updated UI
Shared Data → Context Provider → Descendants

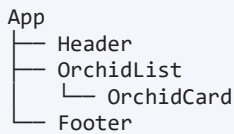
THÔNG TIN BUỔI HỌC SBA301-40 v2 • Session 04/40 • Teaching Session 04/38 • 135 phút • Chapter 04 + phần cốt lõi Chapter 05

1. Tổng quan buổi học

Buổi 04 là bước chuyển quan trọng từ component tĩnh sang component động. Sinh viên sẽ học cách truyền dữ liệu bằng Props, lưu dữ liệu thay đổi bằng State/useState, xử lý sự kiện để cập nhật UI, và nhận diện khi nào Context/useContext có thể giảm prop drilling.

Trọng tâm	Học hôm nay	Mức độ
Props	Parent → Child, read-only, object props, destructuring	CORE
State	State ownership, declarative UI, state update	CORE
useState	State variable, updater, event-driven re-render	CORE
Context/useContext	Prop drilling, Provider, consumer	CORE
Rules of Hooks	Top-level, React function component/custom Hook	CORE
useEffect	Nhận diện side effect/external synchronization	PREVIEW
useReducer	Nhận diện state transitions có cấu trúc	PREVIEW
Custom Hook	Nhận diện reusable Hook-based logic	PREVIEW

1.1. Kết nối với Buổi 03



Ở Buổi 03, các OrchidCard có cấu trúc tốt nhưng dữ liệu còn static/hard-coded. Nếu mỗi Card cần tên, ảnh, category và rating khác nhau, không nên tạo OrchidCard1, OrchidCard2, OrchidCard3. Cần một component tái sử dụng và truyền dữ liệu vào bằng Props.

1.2. Ba câu hỏi trung tâm

Câu hỏi	Khái niệm trả lời
Parent gửi dữ liệu cho Child như thế nào?	PROPS
Component “nhớ” dữ liệu thay đổi như thế nào?	STATE / useState
Nhiều component xa nhau chia sẻ cùng dữ liệu thế nào mà không truyền qua mọi tầng?	CONTEXT / useContext

1.3. Bảng chứng cuối buổi

- Một OrchidCard tái sử dụng nhận object qua Props.
- Một tương tác event-driven sử dụng useState để thay đổi UI.
- Một data-flow trace Parent → Props → Child → UI.
- Một state trace Event → Updater → New State → Render.
- Một quyết định có căn cứ: dùng Context hay chưa cần Context.
- Không vi phạm Rules of Hooks và có Git checkpoint nếu làm trong project.

2. Learning Objectives & Recall

2.1. Learning Objectives

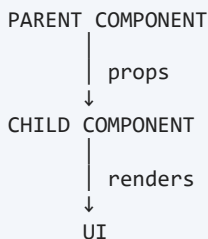
1. Giải thích Props, parent-to-child data flow và lý do Props là read-only.
2. Truyền string, number, boolean và object qua Props; sử dụng destructuring ở mức cơ bản.
3. Dùng một OrchidCard để hiển thị nhiều dữ liệu khác nhau.
4. Giải thích State và phân biệt Props với State dựa trên ownership.
5. Sử dụng useState, đọc current state và cập nhật state bằng updater.
6. Kết nối event handler với state update và giải thích re-render.
7. Giải thích Hook và hai Rules of Hooks nền tảng.
8. Giải thích prop drilling, Context, Provider và useContext.
9. Nhận diện useEffect, useReducer và Custom Hook ở mức preview.
10. Dùng AI để review ownership/data flow, không dùng AI thay thế tư duy thiết kế.

2.2. Recall - không dùng AI ở vòng đầu

11. Component là gì?
12. Component Tree là gì?
13. Parent Component và Child Component là gì?
14. Composition là gì?
15. Vì sao repeated UI là candidate cho component?
16. Import và export dùng để làm gì?
17. JSX là gì?
18. Vì sao OrchidCard là một component hữu ích?

SELF-CHECK Nếu chưa giải thích được component tree và parent/child, hãy xem lại Buổi 03 trước khi học Props. Props chỉ rõ nghĩa khi bạn hiểu component hierarchy.

2.3. Central mental model



USER ACTION → EVENT → STATE UPDATE → RENDER AGAIN → UPDATED UI

SHARED DATA → CONTEXT PROVIDER → Component A / B / C

3. Props - dữ liệu nhận vào của component

3.1. Props là gì?

CORE DEFINITION Props là thông tin được truyền vào một user-defined component thông qua JSX attributes/children. React gom các thông tin này thành đối tượng props.



Child

3.2. Ví dụ Props đầu tiên

Ví dụ

```
// Parent
<OrchidCard orchidName="Taichung Beauty" />

// Child
function OrchidCard(props) {
  return <h2>{props.orchidName}</h2>;
}
```

Trace bắt buộc: "Taichung Beauty" → prop name orchidName → props.orchidName → JSX → Browser UI.

3.3. Props như input

```
Component ≈ Function
Props      ≈ Input
```

```
OrchidCard(data) → UI
```

Đây là analogy để hiểu ownership, không phải cú pháp gọi JavaScript thực tế. Trong React, component được sử dụng bằng JSX syntax.

3.4. Props là read-only

```
// KHÔNG NÊN
props.orchidName = "New Name";
```

QUY TẮC Child nhận Props nhưng không sở hữu dữ liệu đó. Component nhận Props không được tự ý sửa Props.

3.5. One-way data flow

```
Parent
  ↓
Child
  ↓
Grandchild
```

Props thường đi xuống component tree. Điều này làm dependency giữa parent và child rõ ràng hơn.

4. Truyền dữ liệu bằng Props

4.1. Các kiểu dữ liệu thường gặp

Loại	Ví dụ JSX	Ý nghĩa
String	<OrchidCard name="Cattleya" />	String literal dùng dấu nháy
Number	<OrchidCard rating={4.8} />	JavaScript expression dùng {}
Boolean	<OrchidCard isSpecial={true} />	Boolean expression
Object	<OrchidCard orchid={orchid} />	Truyền object để nhóm dữ liệu liên quan

4.2. Object Props - cách dùng phù hợp cho OrchidCard

```
const orchid = {
  id: 1,
  orchidName: "Taichung Beauty",
}
```

```

    category: "Cattleya",
    rating: 4.8
  };

  <OrchidCard orchid={orchid} />

  function OrchidCard({ orchid }) {
    return (
      <Card>
        <Card.Title>{orchid.orchidName}</Card.Title>
        <Card.Text>{orchid.category}</Card.Text>
      </Card>
    );
  }

```

4.3. Destructuring Props

```

function OrchidCard(props) {
  return <h2>{props.name}</h2>;
}

// Tương đương về ý tưởng
function OrchidCard({ name }) {
  return <h2>{name}</h2>;
}

```

Hai cách có cùng conceptual input; destructuring chỉ là cú pháp JavaScript giúp code gọn hơn.

4.4. Reusability với Props

```

<OrchidCard orchid={orchidA} />
<OrchidCard orchid={orchidB} />
<OrchidCard orchid={orchidC} />

```

MENTAL MODEL One Component + Different Props = Reusable UI

4.5. Props trong Class Component - awareness

Tài liệu nguồn của môn còn giới thiệu Class Component. Sinh viên cần nhận diện rằng Class Component truy cập Props bằng `this.props`. Phần thực hành của buổi này dùng Functional Components.

```
this.props.name
```

5. State - dữ liệu component sở hữu và thay đổi

5.1. Vì sao Props chưa đủ?

Props trả lời câu hỏi “data đi vào component như thế nào?”, nhưng chưa giải quyết việc component phải nhớ rằng điều gì đó đã thay đổi sau tương tác người dùng.

[Show Detail]

Hidden → user clicks → Visible

Cần: STATE

5.2. State là gì?

CORE DEFINITION State là dữ liệu được component sở hữu/ghi nhớ, có thể thay đổi theo thời gian và ảnh hưởng đến UI được render.

- isModalOpen
- selectedOrchid
- count
- searchText
- isLoggedIn

5.3. Declarative UI với State

```
State: isDetailOpen = false
UI:    Detail hidden
```

```
State: isDetailOpen = true
UI:    Detail visible
```

Thay vì tự thao tác DOM để “ẩn/hiện” từng phần, React UI được mô tả theo state hiện tại. Khi state thay đổi, component được render lại theo state mới.

5.4. Props vs State - bảng cốt lõi

Props	State
Passed from parent	Owned by component
Input	Internal changing data
Read-only for receiver	Updated through state updater
Configure component	Respond/change over time
External source	Local component memory

CÂU HỎI ĐÚNG Không hỏi “Props hay State cái nào tốt hơn?”. Hãy hỏi: dữ liệu xuất phát ở đâu, ai sở hữu, ai cần, ai có quyền thay đổi?

6. Hook & useState

6.1. Hook là gì?

BEGINNER DEFINITION Hook là React function cho phép Functional Components sử dụng một số khả năng của React.

- useState → state
- useContext → context
- useEffect → external synchronization/side effects (preview)

6.2. Cú pháp useState

```
const [value, setValue] = useState(initialValue);
```

Thành phần	Ý nghĩa
value	current state
setValue	state updater function
initialValue	initial state

6.3. Ví dụ Counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </>
  );
}
```

6.4. State update trace - bắt buộc giải thích được

```
Initial: count = 0
  ↓
User clicks
  ↓
onClick handler
  ↓
setCount(1)
  ↓
React updates state
  ↓
Component renders again
  ↓
UI shows 1
```

6.5. Event-driven UI

User Action → Event → Handler → State Update → Render

Buổi này chỉ cần onClick ở mức nền tảng; chưa cần học toàn bộ DOM event system.

7. State trong OrchidCard & React-Bootstrap Modal

7.1. Local State cho “View Detail”

```
import { useState } from "react";

const [showDetail, setShowDetail] = useState(false);
```

7.2. Kết nối Button → State

```
<Button onClick={() => setShowDetail(true)}>
  View Detail
</Button>
```

7.3. State-driven Modal

```
<Modal
  show={showDetail}
  onHide={() => setShowDetail(false)}
>
  <Modal.Header closeButton>
    <Modal.Title>{orchid.orchidName}</Modal.Title>
  </Modal.Header>
  <Modal.Body>
```

```

      Category: {orchid.category}
    </Modal.Body>
  </Modal>

  OrchidCard
    ↓
  Button
    ↓
  showDetail state
    ↓
  Modal Open / Closed

```

7.4. Predict before click

Câu hỏi	Sinh viên ghi trước khi chạy
Current showDetail	_____
After click	_____
Expected UI	_____

7.5. Không mutate State trực tiếp

```
// Không dùng kiểu thay trực tiếp để thay thế state updater
count = count + 1;
```

RULE Sử dụng React state update mechanism. Với object/array state, không dùng mutation pattern bỏ qua updater.

7.6. State có thể chứa nhiều kiểu dữ liệu

number • string • boolean • object • array

CRUD với state array chưa phải mục tiêu core hôm nay; chỉ cần nhận diện khả năng lưu các kiểu state khác nhau.

8. State Ownership - ai nên sở hữu state?

8.1. Câu hỏi thiết kế

HEURISTIC Component nào cần kiểm soát giá trị đang thay đổi?

Tình huống	State owner hợp lý ở mức cơ bản
Mỗi OrchidCard có Modal riêng	OrchidCard → local showDetail state
Một Modal chung hiển thị selected Orchid	OrchidList/App → state cao hơn trong tree
Một toggle chỉ ảnh hưởng một Card	Giữ state local ở Card
Nhiều sibling cùng cần cùng giá trị	Cân nhắc đưa state lên common parent

8.2. Nguyên tắc Minimal Shared State

- Không đưa local state lên cao nếu chưa có nhu cầu chia sẻ.
- Không dùng Context chỉ vì muốn “có Context trong project”.
- Ưu tiên ownership rõ ràng và data flow dễ trace.
- UI hoạt động chưa đủ; sinh viên phải giải thích ai sở hữu dữ liệu.

8.3. Props/State ownership worksheet nhanh

Value	Nguồn	Owner	Ai cần?	Ai thay đổi?	Props / State / Derived?
orchid	_____	_____	_____	_____	_____
showDetail	_____	_____	_____	_____	_____
rating	_____	_____	_____	_____	_____

9. Context & useContext

9.1. Prop drilling là gì?

```
App
  ↓ theme
Layout
  ↓ theme
Main
  ↓ theme
Toolbar
  ↓ theme
Button
```

Nếu các component trung gian chỉ nhận một prop để chuyển tiếp xuống sâu hơn mà không thực sự sử dụng nó, data flow có thể trở nên bất tiện. Tình huống này thường được gọi là prop drilling.

9.2. Context là gì?

CORE DEFINITION Context là một cơ chế chia sẻ dữ liệu React-side cho descendant components; nó có thể là một alternative khi truyền Props qua nhiều tầng trở nên không phù hợp.

```
Context Provider
  ↓ shared value
Descendant A  Descendant B  Descendant C
```

CRITICAL Context không phải database và không phải persistent storage.

9.3. createContext

```
import { createContext } from "react";

const ThemeContext = createContext(null);
```

createContext tạo context channel/object. Nó chưa tự động phân phối giá trị cho toàn bộ application.

9.4. Provider mental model

```
Provider
  ↓
makes value available
  ↓
descendants
```

9.5. useContext

```
const theme = useContext(ThemeContext);

Context → Provider Value → useContext → Component
```

10. Props hay Context?

10.1. Bảng quyết định

Ưu tiên Props	Cân nhắc Context
Direct parent → child	Nhiều descendants ở xa nhau
Component-specific data	Shared app-level concern
Data flow ngắn, rõ ràng	Prop drilling thực sự trở nên bất tiện
Mặc định cho simple case	Dùng khi có lý do thiết kế rõ ràng

RULE Context không phải “better Props”. Nếu Props đơn giản và rõ ràng thì tiếp tục dùng Props.

10.2. Context micro-scenario

```

App
↓ Provider (current user)
Header      Footer
  
```

Header và Footer cùng cần current user. Đây là ví dụ để minh họa Context. Với application rất nhỏ, dùng Props vẫn có thể hợp lý; điều quan trọng là giải thích trade-off.

10.3. Context verification questions

19. Data có thực sự cần ở nhiều descendants không?
20. Có prop drilling thực sự không?
21. Context có làm ownership khó hiểu hơn không?
22. Có phương án Props đơn giản hơn không?

11. Rules of Hooks

11.1. Hai quy tắc nền tảng

RULE 1 Call Hooks at the top level.

RULE 2 Call Hooks from React function components hoặc custom Hooks.

11.2. Hook trong condition - sai

```

if (loggedIn) {
  const [name, setName] = useState("");
}
  
```

Hook call order phải ổn định. Không đặt Hook trong điều kiện, loop hoặc sau conditional return.

11.3. Hook trong event handler - sai

```

function handleClick() {
  const [count, setCount] = useState(0);
}
  
```

Event handler được phép gọi state updater, nhưng không được tạo Hook bằng cách gọi useState bên trong handler.

11.4. Hook Rule Review checklist

CHECK	Điều kiện hoàn thành
-------	----------------------

☐ PASS ☐ FAIL	Hook được gọi ở top-level của function component/custom Hook.
☐ PASS ☐ FAIL	Hook không nằm trong if/loop/event handler.
☐ PASS ☐ FAIL	Hook import đúng từ React.
☐ PASS ☐ FAIL	Component có tên và cấu trúc phù hợp để React xử lý.

12. Hook Preview - Learn Later

12.1. useEffect - preview

Chapter 05 mô tả useEffect là Hook dùng để kết nối/synchronize component với external system. Buổi này chỉ nhận diện ý tưởng và syntax; data fetching sẽ còn xuất hiện ở các buổi sau.

```
useEffect(() => {
  document.title = "SBA301";
}, []);
```

QUESTION External system ở ví dụ này là document/browser environment. Không dùng useEffect như một “bước bắt buộc sau mỗi render”.

12.2. useReducer - preview

PREVIEW useReducer là một state-management Hook thay thế phù hợp khi state transitions trở nên có cấu trúc/phức tạp hơn. Chưa cần thực hành sâu hôm nay.

12.3. Custom Hook - preview

Repeated Hook-based logic → Custom Hook
Custom Hook name thường bắt đầu bằng use...

Custom Hook là reusable function chứa Hook-based logic; nó không phải React component.

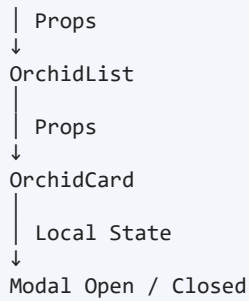
12.4. Core vs Preview Gate

Nếu thời gian thiếu	Ưu tiên
1	Props
2	State
3	useState
4	Hook Rules
5	Context
6	Advanced Hook Preview

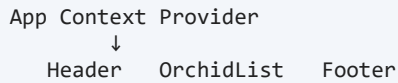
13. Complete Data-flow Model

13.1. Props + State

```
App
|
```



13.2. Context



13.3. Cách trace một vấn đề dynamic UI



DEBUGGING HABIT Khi UI không update, trace data flow thay vì chỉ nhìn vào JSX hoặc yêu cầu AI viết lại toàn bộ component.

14. Zero-Assumption Lab - Make Orchid Gallery Reusable and Interactive

Starting point: Orchid Gallery của Session 03. Goal: Static Cards → Reusable Cards using Props → Interactive Detail using State → Optional Shared App Data using Context.

Step 0 - Baseline

```
npm run dev
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Orchid Gallery loads.
☐ PASS ☐ FAIL	Cards visible.
☐ PASS ☐ FAIL	Không có relevant console errors.

Step 1 - Tạo sample data

```
const orchid = {
  id: 1,
  orchidName: "Taichung Beauty",
  image: "...",
  category: "Cattleya",
  isSpecial: true,
  rating: 4.8
};
```

Nếu Lab chính thức có schema Orchid cụ thể, sử dụng schema của Lab thay cho ví dụ minh họa.

Step 2 - Pass Orchid as Prop

```
<OrchidCard orchid={orchid} />
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Component compile không lỗi.
☐ PASS ☐ FAIL	Prop name đúng là orchid.

Step 3 - Read Prop trong Child

```
function OrchidCard({ orchid }) {
  // render name, image, category, rating
}
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Tên hiển thị đúng.
☐ PASS ☐ FAIL	Ảnh hiển thị đúng.
☐ PASS ☐ FAIL	Category/rating hiển thị đúng.

15. Lab - Reuse, State, Event

Step 4 - Multiple Orchids, One Component

```
<OrchidCard orchid={orchidA} />
<OrchidCard orchid={orchidB} />
<OrchidCard orchid={orchidC} />
```

CHECK	Điều kiện hoàn thành
-------	----------------------

☐ PASS ☐ FAIL	Chỉ có một OrchidCard definition.
☐ PASS ☐ FAIL	Mỗi Card hiển thị dữ liệu khác nhau.

Step 5 - Explain read-only Props

Trả lời bằng lời: Nếu OrchidCard muốn đổi orchid.orchidName, có nên sửa trực tiếp object nhận từ Parent không? Vì sao?

EXPECTED IDEA Không. Child không sở hữu Props; component nhận Props không tự ý sửa dữ liệu nhận vào.

Step 6 - Add Local State

```
const [showDetail, setShowDetail] = useState(false);
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Đã import useState.
☐ PASS ☐ FAIL	Component compile.
☐ PASS ☐ FAIL	Initial state = false.

Step 7 - Connect Button

```
onClick={() => setShowDetail(true)}
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Click event thực sự chạy.
☐ PASS ☐ FAIL	Updater setShowDetail được gọi.

Step 8 - Show/Hide Detail

```
show={showDetail}
onHide={() => setShowDetail(false)}
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Button mở detail/modal.
☐ PASS ☐ FAIL	Close action đóng detail/modal.
☐ PASS ☐ FAIL	UI phụ thuộc vào current state.

Step 9 - Predict before Click

Trước khi chạy	Điền
Current showDetail	_____
After click	_____
Expected UI	_____

16. Lab - Context Micro-Exercise & Verification

Step 10 - Optional simple counter

Nếu đủ thời gian, tạo Likes/Views counter bằng useState để củng cố event → updater → state → re-render. Nếu thời gian ngắn, bỏ bước này để giữ trọng tâm Props/State/Context.

Step 11 - Context Micro-Exercise

Chọn một shared value đơn giản: course name, current user hoặc theme. Không đưa toàn bộ Orchid data vào Context chỉ để “cho có”.

```
App
  ↓ Provider
Header
Footer
```

Step 12 - Context Verification

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Header đọc shared value.
☐ PASS ☐ FAIL	Footer đọc shared value.
☐ PASS ☐ FAIL	Không cần direct prop chain chỉ để chuyển tiếp value.
☐ PASS ☐ FAIL	Sinh viên giải thích được: Context có thật sự cần hay chỉ là demo giáo dục.

Step 13 - Git Checkpoint

```
git status
git add .
git commit -m "Add props state and context fundamentals"
git log -1
```

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Working tree được kiểm tra trước commit.
☐ PASS ☐ FAIL	Commit message có nghĩa.
☐ PASS ☐ FAIL	git log -1 hiển thị commit mới.

Lab Completion Gate

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Reusable component dùng Props.
☐ PASS ☐ FAIL	Object prop được render đúng.
☐ PASS ☐ FAIL	useState dùng đúng Rules of Hooks.
☐ PASS ☐ FAIL	Event-driven interaction thay đổi UI.
☐ PASS ☐ FAIL	Props không bị mutate.
☐ PASS ☐ FAIL	Context chỉ dùng nếu có lý do hoặc được ghi rõ là micro-demo.
☐ PASS ☐ FAIL	Console không có relevant error.

17. Trace Worksheets

17.1. Props Trace Worksheet

Field	Student answer
VALUE	Taichung Beauty
DEFINED IN	_____
PASSED BY	_____
PROP NAME	_____
RECEIVED BY	_____
RENDERED AT	_____

17.2. State Trace Worksheet

Field	Student answer
STATE NAME	_____
INITIAL VALUE	_____
USER EVENT	_____
UPDATER	_____
NEW VALUE	_____
UI CHANGE	_____

17.3. Context Trace Worksheet

Field	Student answer
CONTEXT	_____
PROVIDER	_____
VALUE	_____
CONSUMER	_____
HOOK	_____
WHY CONTEXT	_____

17.4. Explain in 60 seconds

Data Source → Parent → Props → Child → UI
 Button Click → Handler → Updater → State → Re-render
 Context Provider → useContext → Consumer

18. AI-Augmented Learning - Human First

18.1. AI-0 Activity

Không dùng AI ở vòng đầu. Scenario: ProductCard cần name, price, image; người dùng có thể Add Favorite. Hãy xác định giá trị nào nên là Props, giá trị nào nên là State, và component nào sở hữu chúng.

Candidate	Expected reasoning
name / price / image	Props candidate - dữ liệu cấu hình của ProductCard
isFavorite	Local State candidate - thay đổi do user interaction, nếu chỉ ảnh hưởng Card

18.2. AI Props vs State Review Prompt

Act as a React tutor.

Component requirement:
[REQUIREMENT]

My design:
Props: [...]
State: [...]

Do NOT generate code.
Check whether each value should be:
Prop / Local State / Shared Context / Derived value.
Explain ownership and which component should control it.
Keep scope at beginner SBA301 level.

18.3. AI Data-flow Review Prompt

Review my React component tree and data flow:
[TREE]
[DATA FLOW]

Do not redesign it first.
Trace:
Data Source → Parent → Props → Child → State → UI
Identify the FIRST unclear or duplicated ownership decision.

19. AI Review for Hooks, State & Context

19.1. Hook Rule Review

Review this React component:
[CODE]

Do not rewrite it.
Check only the Rules of Hooks.
For each Hook call tell me:
Top-level? Inside React function component? Inside condition? Inside loop? Inside handler?
Classify: VALID / INVALID / NEEDS CONTEXT.

19.2. State Debug Prompt

My React UI does not update.
Expected: [EXPECTED]
Actual: [ACTUAL]
Initial state: [...]

```
Event: [...]
Updater: [...]
Component: [CODE]
Browser Console: [...]
```

Do not rewrite the component.

Trace: Event → Handler → State Updater → New State → Rendered JSX.

Find the FIRST broken stage.

19.3. Context Review Prompt

I am using React Context for: [DATA]
Component tree: [TREE]

Do not add Context automatically.

Evaluate whether:

1. direct props are enough;
2. prop drilling is actually occurring;
3. Context is justified;
4. state ownership remains clear.

Recommend the simplest option.

19.4. AI Hallucination Check

CLAIM TO CHECK “useState automatically saves state into SQL Server.”

Expected correction: useState là client/browser component state. Database persistence sẽ thuộc flow HTTP → Spring Boot → Repository → DB ở phần full-stack sau này.

19.5. Overengineering Guard

Use the smallest state mechanism appropriate for this requirement.

Prefer local state unless evidence shows shared state is required.

Không biến isModalOpen thành Redux/Zustand/global store chỉ vì AI đề xuất “kiến trúc chuyên nghiệp”.

20. Common Misconceptions & Errors

20.1. Misconceptions cần loại bỏ

- Props và State là một.
- Props thuộc sở hữu của Child.
- Child nên sửa Props nhận được.
- Mọi value đều phải là State.
- useState lưu dữ liệu vào database.
- Setter thay đổi toàn bộ app state globally.
- State đôi khi cần manual DOM manipulation.
- useState có thể gọi trong bất kỳ function nào.
- Hooks có thể gọi trong condition.
- Context là database.
- Context nên thay mọi Props.
- Context tự persistent.
- useContext tạo Context.
- createContext tự phát value cho cả app.
- useEffect bắt buộc cho mọi component.
- useReducer chính là Redux.
- Custom Hook là React component.
- Props chỉ có thể là string.

- State chỉ có thể là number.

20.2. Common Errors Table

Symptom	Likely Cause	Evidence	Minimal Fix	Verify
prop undefined	Prop name mismatch / chưa truyền	Inspect Parent JSX + Child destructuring	Đồng bộ prop name	Child render đúng
wrong destructuring	Tên field/object sai	Console + code	Kiểm tra shape object	Value xuất hiện
setter chạy trong render	Gọi setX() ngay khi render	Repeated render/error	Chuyển vào event handler	Không loop render
click không update	Handler/updater sai	Console/log	Trace Event→Handler→Updater	State đổi
Hook inside condition	Vi phạm Hook Rules	Code inspection	Đưa Hook lên top-level	Rule pass
missing Hook import	useState/useContext undefined	Compile error	Import từ react	Compile pass
consumer outside Provider	Context value null/default	Component tree	Đặt Provider đúng ancestor	Consumer đọc value
Modal not closing	onHide/updater sai	State trace	setShow(false)	Modal đóng

21. Debugging Decision Tree

```

Dynamic UI fails
↓
Is data correct in parent?
↓
Is prop passed?
↓
Is prop name correct?
↓
Does child receive it?
↓
Is state initialized correctly?
↓
Does event fire?
↓
Does state updater run?
↓
Does JSX depend on state?
↓
If shared data:
Provider present?
↓
useContext reading correct Context?

```

DEBUG RULE Sửa một lớp lỗi tại một thời điểm. Không đổi đồng thời Props, State, JSX và Context rồi mới chạy lại.

21.1. Evidence trước khi hỏi AI

- Expected result.
- Actual result.
- Current Props.
- Initial State.
- Event handler.
- State updater.
- First meaningful browser console error.
- Relevant component code.
- Component tree/Provider position nếu dùng Context.

22. Real-world Examples & Software Engineering Connection

22.1. Mapping theo domain

Domain	Props	State	Context candidate
E-commerce	Product name, price, image	Quantity, isFavorite	Current User, Theme
Hotel	Room information	Selected date, showModal	Authenticated user
News	Article info	Expanded/collapsed	Logged-in account

22.2. Software Engineering Connection

Nguyên tắc	Áp dụng trong React
Data Ownership	Ai sở hữu value?
Single Responsibility	Không trộn state không liên quan.
Explicit Data Flow	Props làm dependency nhìn thấy được.
Minimal Shared State	Không globalize local state.
Predictability	UI follows data/state.

22.3. Foundation vs Professional Practice

SBA301 Foundation	Professional React Awareness
local useState	deliberate state architecture
direct props	typed contracts where applicable
Context basics	scoped shared state
simple events	domain interactions
basic Hook rules	lint-enforced Hook rules
simple state	reducers for complex transitions

Cột Professional Practice chỉ để awareness, không phải yêu cầu bắt buộc của Buổi 04 và không được dùng để overengineer bài tập.

23. Project-Based Learning Transfer

23.1. Review component tree của nhóm

Mỗi nhóm lấy component tree từ Session 03 và đánh dấu cho từng component: Props Needed, State Needed, Shared Data Candidate.

Component	Props	Local State	Context Candidate
NewsCard	article	expanded	—
Header	—	menuOpen	currentUser
RoomCard	room	showDetail	—
Nhóm tự điền	_____	_____	_____

23.2. Project minimum requirement

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Ít nhất 1 reusable component dùng Props.
☐ PASS ☐ FAIL	Ít nhất 1 component dùng useState.
☐ PASS ☐ FAIL	Ít nhất 1 event-driven UI interaction.
☐ PASS ☐ FAIL	Context chỉ dùng nếu có shared-data case hợp lý.

VALID EVIDENCE Nếu Context chưa cần, nhóm có thể ghi: “Context Not Needed Yet - Reason: ...”. Đây là bằng chứng thiết kế hợp lệ.

23.3. Group Review Questions

23. Dữ liệu bắt nguồn ở đâu?
24. Component nào sở hữu?
25. Component nào chỉ nhận Props?
26. Value nào thay đổi theo user interaction?
27. State có đang bị đưa lên quá cao không?
28. Có prop drilling thật sự chưa?
29. Nếu dùng Context, tại sao Props không còn phù hợp?

24. Human Verification Gate - không dùng AI

30. Props là gì?
31. Ai sở hữu Props?
32. Vì sao Props là read-only?
33. One-way data flow là gì?
34. Parent truyền object cho child như thế nào?
35. Props và State khác nhau thế nào?
36. State là gì?
37. useState là gì?
38. Hai giá trị trả về bởi useState có ý nghĩa gì?
39. Event nào trigger state update trong Lab?
40. Tại sao UI update sau khi state thay đổi?
41. Có nên mutate State trực tiếp không?
42. Hook là gì?
43. Nêu hai Rules of Hooks.
44. Có gọi Hook trong condition được không?
45. Context là gì?
46. Prop drilling là gì?
47. Context khác Props ở đâu?
48. useContext làm gì?
49. Context có persist vào database không?
50. useEffect dùng để làm gì ở mức conceptual?
51. useReducer là gì ở mức awareness?
52. Custom Hook là gì?
53. Component nào nên sở hữu showModal?
54. Vì sao local state nên giữ local khi có thể?

PASS CRITERION Ứng dụng chạy nhưng sinh viên không trace được Props/State/Context thì chưa chứng minh được data-flow understanding.

24.1. Result Analysis Sheet

Area	Evidence
Reusable component	_____
Props passed	_____
Parent	_____
Child	_____
State variable	_____
Initial value	_____
Updater	_____
Trigger	_____
UI change	_____
Context used	YES / NO
Provider / Consumer / Shared value	_____
If NO - reason	_____
Hook violation found/corrected	_____

25. Tips, Exit Ticket & Reflection

25.1. Tips

Tip	Nhớ
Props Tip	Ask: Who owns this data?
State Tip	Store only what needs to change and affect UI.
Context Tip	Use only when sharing actually becomes awkward.
Hook Tip	Hooks at top level.
Debugging Tip	Trace data instead of staring at JSX.
AI Tip	Ask “Who should own this value?” before “Write the code.”

25.2. Exit Ticket

55. Vì sao `<OrchidCard orchid={orchid1}/>` và `<OrchidCard orchid={orchid2}/>` thể hiện tính reusable?
56. Điền: `const [show, _____] = useState(false);`
57. Trace: Button Click → ? → State → ?
58. Khi nào nên cân nhắc Context thay vì chỉ Props?
59. Vì sao `if (condition) { useState(0); }` là không hợp lệ?

25.3. Answer Key - tự đối chiếu sau khi nộp

- Một component definition nhận different Props nên hiển thị được dữ liệu khác nhau.
- `setShow`.
- Event Handler → State → Render/UI update (chi tiết hơn: handler → updater → new state → re-render).
- Khi dữ liệu thực sự cần bởi nhiều descendant và prop drilling trở nên bất tiện; Context phải có lý do rõ ràng.

- Vì Hook phải được gọi ở top-level; condition có thể làm thay đổi Hook call order.

25.4. Reflection

- Before today I thought Props were...
- Now I understand State as...
- The owner of my state is...
- My UI changes when...
- One case where Context would be useful is...
- One case where Context would be unnecessary is...
- AI suggested...
- I verified the data flow by...

26. Home Practice & Strong-student Challenge

26.1. Home Practice

60. Convert một repeated component trong group project sang Props.
61. Add một local interaction dùng useState: show/hide detail, favorite, counter hoặc toggle.
62. Vẽ Data Source → Parent → Props → Child.
63. Vẽ state transition Initial → Event → Updated State → UI.
64. Xác định Context có cần hay không và giải thích trong 3 câu.

26.2. Optional Strong-student Challenge

Dùng một useEffect đơn giản để update document title dựa trên state value. Sinh viên phải trả lời: external system nào đang được synchronize? Không dùng API fetching trong challenge này.

```
useEffect(() => {
  document.title = `SBA301 - ${someState}`;
}, [someState]);
```

26.3. Chuẩn bị cho Session 05

```
Component Architecture
+
React-Bootstrap
+
Props
+
State
+
Hooks
+
Orchid Gallery

→ INTEGRATED REACT LAB 01
```

Session 05 là buổi consolidation: tích hợp các kiến thức React đã học vào một mini-application hoàn chỉnh trước khi chuyển sang React Router & Navigation.

27. Kế hoạch 135 phút

Thời gian	Nội dung	Evidence
00–10	Recall Component Architecture	Trả lời nhanh / component tree
10–30	Props, read-only, Parent → Child	Props trace
30–45	Props exercise, object props, destructuring	Reusable OrchidCard

Thời gian	Nội dung	Evidence
45–62	State mental model, declarative UI	Props vs State explanation
62–78	useState, events, state update trace	Event→Updater→Render
78–94	Hands-on: Props + Modal State	Interactive detail
94–104	Context, prop drilling, useContext	Context decision
104–112	Rules of Hooks	Invalid/valid recognition
112–118	useEffect/useReducer/Custom Hook preview	Concept recognition
118–127	AI-0 + AI data-flow review	Human-first decision + review
127–132	Project transfer	Props/State/Context matrix
132–135	Human Verification + Exit Ticket	5-question exit ticket

27.1. Priority nếu thiếu thời gian

Props > State > useState > Hook Rules > Context > Advanced Hook Preview

27.2. Learning transition

SESSION 03: I can structure UI.

↓

SESSION 04: I can make component data flow and change.

↓

SESSION 05: I can integrate these ideas into a working React application.

28. Fundamental Glossary

Term	Definition	Core Meaning	Example	Why Important	Scope
Props	Data passed to component	Input from parent	orchid={orchid}	Reusable components	NOW
Read-only	Receiver should not mutate value	Child does not own Props	props.name	Predictable flow	NOW
Parent Component	Component rendering child	Source/owner may pass data	OrchidList	Data source	NOW
Child Component	Component rendered by parent	Receives Props	OrchidCard	Composition	NOW
One-way Data Flow	Data flows down component tree	Parent → Child	App→List→Card	Traceability	NOW
State	Changing data owned by component	Local memory	showDetail	Interactive UI	NOW
State Variable	Current state value	Read current value	showDetail	Render condition	NOW
State Updater	Function to request state update	Change state correctly	setShowDetail	Triggers update	NOW
Initial State	Starting state value	Initial UI condition	false	Predict first UI	NOW
Re-render	Component renders again after relevant update	UI recalculation	Modal appears	Dynamic UI	NOW
Event	User/system interaction signal	Trigger	click	Interaction	NOW

Term	Definition	Core Meaning	Example	Why Important	Scope
Event Handler	Function reacting to event	Calls updater	handleClick	Connect UI→state	NOW
Hook	React function for capabilities	Functional component API	useState	Modern React foundation	NOW
useState	State Hook	Create/update local state	useState(false)	Interactive components	NOW
Rules of Hooks	Constraints on Hook calls	Stable call order	top-level	Correctness	NOW
Context	Shared data access mechanism	Avoid excessive prop drilling	UserContext	Shared concerns	NOW
createContext	Creates context object/channel	Define context	createContext(null)	Context setup	NOW
Provider	Makes context value available	Ancestor supplies value	Provider	Sharing	NOW
useContext	Reads/subscribes to Context	Consumer Hook	useContext(UserContext)	Consume shared value	NOW
Prop Drilling	Passing props through intermediate layers	Too many forwarding layers	App→...→Button	Motivation for Context	NOW
useEffect	Effect Hook	External synchronization	document.title	Side effects	PREVIEW
Side Effect	Interaction outside pure render calculation	External system work	title/API/subscription	Effect reasoning	PREVIEW
Dependency Array	Controls effect dependencies	Effect timing input	[someState]	useEffect reasoning	PREVIEW
useReducer	Reducer Hook	Structured state transitions	dispatch/action	Complex state	PREVIEW
Custom Hook	Reusable Hook-based function	Share logic	useSomething	Reuse logic	PREVIEW
State Ownership	Who controls changing value	Design decision	Card owns showDetail	Maintainability	NOW
Derived Data	Value calculated from other data	May not need state	total from items	Avoid redundant state	AWARENESS

29. Final Student Checklist

CHECK	Điều kiện hoàn thành
☐ PASS ☐ FAIL	Tôi phân biệt được Props, State và Context bằng ownership/use case.
☐ PASS ☐ FAIL	Tôi truyền được object Props và destructure đúng.
☐ PASS ☐ FAIL	Tôi biết Props là read-only.
☐ PASS ☐ FAIL	Tôi tạo được local state bằng useState.
☐ PASS ☐ FAIL	Tôi trace được Event → Handler → Updater → State → Re-render.
☐ PASS ☐ FAIL	Tôi không gọi Hook trong condition/loop/event handler.
☐ PASS ☐ FAIL	Tôi giải thích được prop drilling.
☐ PASS ☐ FAIL	Tôi chỉ dùng Context khi có lý do.
☐ PASS ☐ FAIL	Tôi biết useEffect/useReducer/Custom Hook là preview, không

	overuse.
☐ PASS ☐ FAIL	Tôi hoàn thành Orchid mini-lab hoặc equivalent project feature.
☐ PASS ☐ FAIL	Tôi có evidence: browser, console, code, trace worksheet, Git checkpoint.
☐ PASS ☐ FAIL	Tôi có thể giải thích toàn bộ flow mà không cần AI.

SESSION 04 COMPLETE Khi tất cả mục trên PASS, bạn đã sẵn sàng cho Integrated React Lab 01 ở Session 05.